TABLE OF CONTENT

Chapter 1 — Introduction

1.1Background

In recent years, digital data generation has increased rapidly due to smartphones, high-resolution cameras, IoT devices, and cloud-based applications. Personal users now generate large amounts of photos, videos, documents, backups, and media files on a daily basis. Traditionally, users depend on centralized cloud storage platforms such as **Google Drive**, **Dropbox**, and **OneDrive** for storing and accessing this data. These platforms offer convenience, synchronization, and remote accessibility. However, they also introduce several concerns related to privacy, subscription costs, storage limitations, and reliance on third-party infrastructure.

A major drawback of public cloud systems is the lack of complete ownership. User data remains stored on infrastructure controlled by external companies. Even though these providers implement security standards, users must trust that their personal files, media, and sensitive information are handled responsibly. In addition, recurring subscription fees for larger storage capacities can become expensive over time. As data size grows, users often need to upgrade to higher storage plans, increasing long-term costs.

This has led to growing interest in **self-hosted infrastructure**, where users build and manage their own servers for storage and services. Self-hosting allows complete control over hardware, software, networking, security policies, and data access. Instead of depending entirely on third-party cloud providers, users can deploy their own private cloud environment inside a home or office network.

Among modern low-power computing devices, the Raspberry Pi 5 has emerged as a highly capable platform for self-hosted services. Compared to earlier Raspberry Pi generations, Raspberry Pi 5 provides significantly improved CPU performance, memory bandwidth, I/O throughput, and storage compatibility. With support for 8 GB RAM, high-speed interfaces, and Gigabit Ethernet, it can efficiently handle lightweight virtualization, containerized applications, storage management, and network services. Its compact size, low power consumption, and affordability make it an ideal candidate for building personal server infrastructure.

This project focuses on developing a **self-hosted NAS (Network Attached Storage) and secure personal cloud ecosystem using Raspberry Pi 5**. The proposed system is not limited to file storage; it integrates multiple services including DNS filtering, recursive DNS resolution, remote secure access, media streaming, automatic photo backup, backup synchronization, and future extensibility toward home automation and development environments.

The system uses CasaOS as the primary management platform. CasaOS provides a user-friendly graphical interface that simplifies deployment and management of containerized applications. Unlike traditional command-line-only server administration, CasaOS enables app installation, service monitoring, storage management, and container orchestration through a clean dashboard interface. This makes advanced self-hosted infrastructure more accessible even for users with limited Linux administration experience.

Containerization plays a central role in the project. Applications are deployed through Docker containers, ensuring service isolation, easy updates, consistent environments, and efficient resource utilization. Docker eliminates dependency conflicts between applications and allows the server to run multiple independent services simultaneously.

To improve network security and browsing experience, the project integrates Pi-hole and Unbound. Pi-hole acts as a network-wide DNS sinkhole, blocking advertisements, trackers, telemetry, and malicious domains before they reach client devices. Instead of installing ad blockers on individual devices, a single Pi-hole deployment protects the entire network. Unbound complements this by acting as a recursive DNS resolver. Rather than relying on public DNS providers, Unbound resolves DNS queries directly from root servers, enhancing privacy and reducing third-party dependency.

Secure remote access is another critical requirement. Many self-hosted systems expose ports directly to the internet, increasing attack surface and security risks. This project avoids direct port forwarding by using Cloudflare Tunnel and Cloudflare Access. Cloudflare Tunnel securely connects internal services to a public domain without exposing the local network. Cloudflare Access adds Zero Trust authentication policies, allowing only authorized users to access services through email verification and session controls. Unauthorized requests are denied before reaching the server. Session expiration after 30 minutes further improves security.

For secure private networking, the system also uses Tailscale. Built on WireGuard technology, Tailscale creates an encrypted mesh VPN between devices. This allows secure access to the NAS, SSH terminal, and internal services from anywhere without exposing ports publicly. Tailscale simplifies remote management while maintaining strong encryption.

Beyond infrastructure and security, the server hosts practical personal applications. Media services such as Jellyfin and Navidrome transform the NAS into a personal entertainment server. Jellyfin enables self-hosted video and media streaming, while Navidrome provides music streaming similar to commercial services but with complete ownership of the music library.

For automatic photo and media backup, the project uses Immich. Immich offers features comparable to commercial photo backup solutions, including automatic upload, gallery management, and indexing. This allows smartphone photos and videos to be backed up directly to private infrastructure instead of external cloud services.

Backup and redundancy are also considered. Using **Syncthing**, important files can be synchronized across multiple devices for backup purposes. Additionally, high availability planning includes a secondary **Raspberry Pi Zero node** capable of acting as a fallback for essential services such as Pi-hole and Tailscale.

This project demonstrates that modern self-hosted infrastructure is no longer limited to enterprise-grade servers. With proper architecture, security design, and efficient software selection, a small low-power device like Raspberry Pi 5 can provide enterprise-inspired capabilities such as private cloud storage, Zero Trust networking, secure remote access, DNS security, media hosting, and service orchestration.

1.2 Problem Statement

Modern users face several challenges with centralized cloud platforms:

1. Limited free storage
2. Expensive subscription upgrades
3. Privacy concerns
4. Dependence on third-party infrastructure
5. Limited customization
6. Lack of full data ownership

Commercial NAS systems such as those from **Synology** and **QNAP** provide advanced features but often involve high initial costs and proprietary ecosystems.

Therefore, a need exists for a low-cost, scalable, secure, and customizable self-hosted storage solution.

1.3 Objectives of the Project

Main objectives include:

- Design a private NAS using Raspberry Pi 5
- Implement centralized storage using SSD
- Provide secure remote access
- Eliminate dependency on public cloud providers
- Implement network-wide ad blocking
- Improve DNS privacy
- Enable media streaming
- Support automatic photo backup
- Provide scalable infrastructure for future expansion
- Maintain low power consumption

1.4 Scope of the Project

The scope of this project includes:

Storage Services

- File storage
- Backup storage
- Media storage

Security Services

- VPN access
- Zero Trust authentication
- DNS filtering
- Recursive DNS

Application Hosting

- Media streaming
- Music streaming
- Photo backup
- Cloud workspace

Future Expansion

- Home automation
- Development stack
- Website hosting
- Database hosting
- Private authentication system

1.5 Advantages of Proposed System

The proposed NAS provides multiple benefits:

Cost Efficiency

Compared to commercial NAS systems, Raspberry Pi 5 reduces hardware cost significantly.

Low Power Consumption

Traditional servers consume 100–400W. Raspberry Pi 5 typically consumes much less, making 24/7 operation practical.

Privacy

All files remain under user control.

Customization

Users can install and configure any compatible service.

Scalability

Docker-based deployment makes future expansion easier.

Chapter 2 — Literature Review

2.1 Traditional Cloud Storage

Cloud storage platforms dominate consumer storage due to convenience. These services provide:

- Synchronization
- Backup
- Sharing
- Collaboration

However, limitations include:

- subscription costs
- vendor lock-in
- limited customization
- privacy concerns

2.2 Commercial NAS Systems

Commercial NAS solutions provide:

- RAID
- Backup
- App ecosystem
- Media streaming

Advantages:

- polished UI
- easy setup
- vendor support

Disadvantages:

- expensive
- proprietary
- limited flexibility

2.3 Self-hosted Infrastructure Trend

Recent growth in open-source software has accelerated self-hosting. Tools like:

- Docker
- Casa OS
- Pi-hole
- Tailscale

have reduced deployment complexity.

Self-hosting is increasingly popular among:

- Developers
- privacy enthusiasts
- homelab users
- small businesses

Chapter 3 — Hardware and Software Requirements

3.1 Hardware Requirements

The proposed NAS infrastructure is built primarily around the Raspberry Pi 5. Hardware selection was based on performance, power efficiency, storage expandability, and 24/7 reliability. Unlike traditional desktop servers, this setup achieves server-grade functionality while maintaining low power consumption and compact physical dimensions.

The hardware components used in this project are listed below.

Component	Specification	Purpose
Main Compute Node	Raspberry Pi 5 (8GB RAM)	Core server
Storage	SSD via NVMe-to-SATA HAT	High-speed persistent storage
Network	Gigabit Ethernet	Stable LAN connectivity
Power Supply	Official 27W USB-C	Reliable power delivery
Secondary Node	Raspberry Pi Zero 2WH	High availability backup
Cooling	Active cooling fan/heatsink	Thermal management
Router	DNS configurable router	Network DNS routing

3.2 Raspberry Pi 5 as Server Platform

The Raspberry Pi 5 acts as the central processing unit of the entire NAS infrastructure. Compared to Raspberry Pi 4, the Pi 5 offers major improvements in CPU performance, I/O bandwidth, memory subsystem, and storage handling.

Important reasons for choosing Raspberry Pi 5 include:

High CPU Performance

Pi 5 uses a quad-core ARM Cortex-A76 architecture, significantly faster than earlier Pi generations. This allows simultaneous handling of:

- Docker containers
- DNS filtering
- VPN services
- media streaming
- backup services

The improved CPU makes container-heavy workloads practical.

8GB RAM Configuration

Your system uses the **8GB RAM variant**, allowing comfortable multitasking across many services.

Current idle utilization:

- CPU ≈ 1%
- RAM ≈ 20%

This indicates large headroom for future expansion.

This free memory can support:

- additional containers
- databases

- automation systems
- monitoring tools
- development environments

Gigabit Networking

NAS performance depends heavily on network throughput. Raspberry Pi 5 supports Gigabit Ethernet, allowing significantly faster transfer speeds compared to Wi-Fi-only solutions.

Benefits include:

- reduced latency
- better file transfer
- smoother streaming
- improved backup reliability

3.3 Storage Architecture

Storage is the backbone of any NAS system. Traditional microSD cards are not ideal for long-term server workloads because of limited write endurance and slower random I/O.

Therefore, this project uses an **SSD connected via NVMe-to-SATA expansion HAT**. Storage architecture:

SSD

↓

NVMe-to-SATA Expansion HAT

↓

Raspberry Pi 5

↓

Linux File system

↓

Docker Volumes / NAS Storage

Advantages of SSD storage:

Faster Read/Write Operations

Compared to SD cards, SSDs provide:

- faster boot
- lower latency
- better sequential transfers
- superior random I/O

This improves:

- media scanning
- database performance
- photo indexing

- backups

Better Reliability

Server workloads generate frequent read/write cycles. SSD endurance makes them much more suitable than SD cards.

Improved Container Performance

Applications like:

- Immich
- databases
- Jellyfin metadata indexing

perform better on SSD storage.

3.4 Secondary Node for High Availability

A Raspberry Pi Zero 2W is included in planning as a secondary node.

Primary roles of secondary node:

- backup Pi-hole service
- backup Tailscale access
- emergency DNS failover
- lightweight monitoring

This creates redundancy.

Example scenario:
If main Pi fails due to:

- power issue
- overheating
- OS corruption
- storage issue

critical services can remain available.

High availability is especially important for:

- DNS resolution
- Network wide Ad Blocking
- remote access
- network stability

3.5 Power and Thermal Considerations

A NAS server runs 24/7, so thermal and power stability are essential.

Power Efficiency

Traditional servers:

- 100W–400W+

Raspberry Pi 5:

- dramatically lower power consumption

Advantages:

- lower electricity cost
- silent operation
- always-on deployment

Thermal Management

Heavy workloads like:

- video transcoding
- large backups
- container compilation

increase CPU temperature.

Cooling system includes:

- heatsink
- active fan

Benefits:

- prevents thermal throttling
- improves lifespan

3.6 Software Requirements

The software stack is designed around modular container-based infrastructure.

Software	Role
Raspberry Pi OS Lite	Base operating system
Docker	Container runtime
CasaOS	Management interface
Pi-hole	DNS filtering
Unbound	Recursive DNS
Tailscale	Secure VPN
Cloudflare Tunnel	Remote exposure
Cloudflare Access	Authentication
Jellyfin	Media streaming
Navidrome	Music streaming
Immich	Photo backup
Syncthing	Backup synchronization

3.7 Operating System

The base operating system is **Raspberry Pi OS Lite 64-bit**.

Why Lite version?

Because Lite excludes unnecessary desktop packages, reducing:

- RAM usage
- storage usage
- CPU overhead

Advantages:

- lightweight
- efficient
- optimized for servers
- improved stability

This creates more available resources for applications.

3.8 Docker Containerization

CasaOS serves as the management dashboard.

CasaOS simplifies:

- app installation
- storage browsing
- container management
- service deployment

Key features:

App Store

One-click installation of many self-hosted apps.

Examples:

- Jellyfin
- Immich
- Nextcloud

User Interface

Clean and beginner-friendly.

Unlike terminal-only management, CasaOS allows easier control through browser.

Container Integration

Directly interfaces with Docker.

Storage Browser

Files can be managed visually.

Service Dashboard

Shows running services and health.

This greatly improves usability.

Chapter 4 — System Architecture

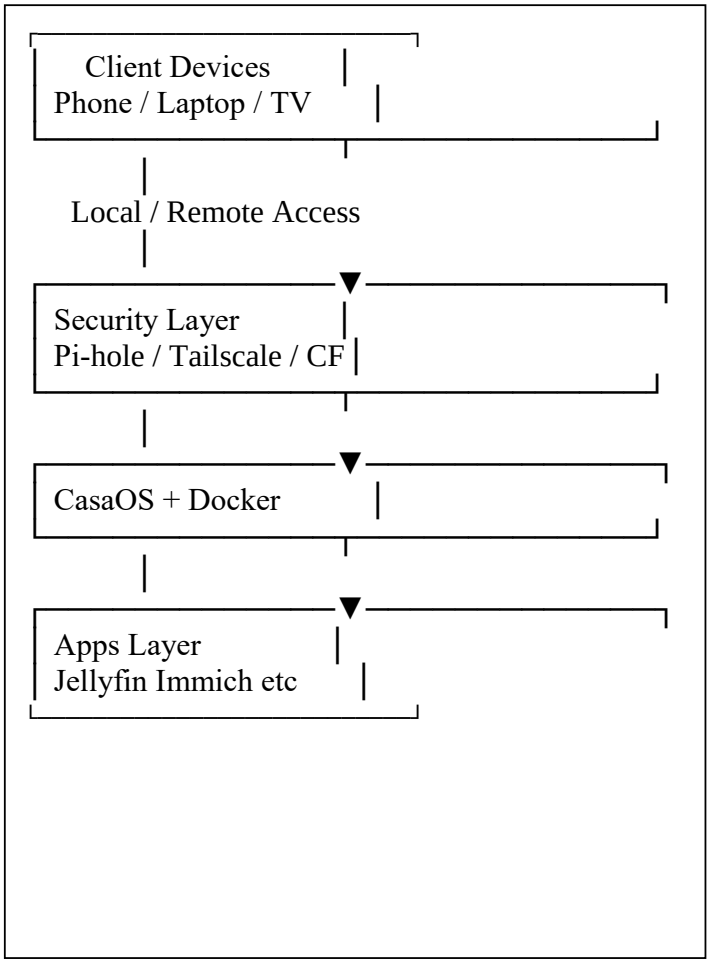
4.1 Architectural Overview

The system follows a layered architecture for modularity, maintainability, and security.

Layers include:

1. Hardware Layer
2. OS Layer
3. Container Layer
4. Security Layer
5. Application Layer

Architecture diagram:



4.2 Access Model

Users can access services through two paths:

Local Network Access

Inside home network:

- direct IP access
- low latency
- high speed

Example:

192.168.x.x:port

Useful for:

- large file transfer
- streaming
- admin tasks

Remote Access

Outside home network:

- Tailscale VPN
  or
- Cloudflare Tunnel

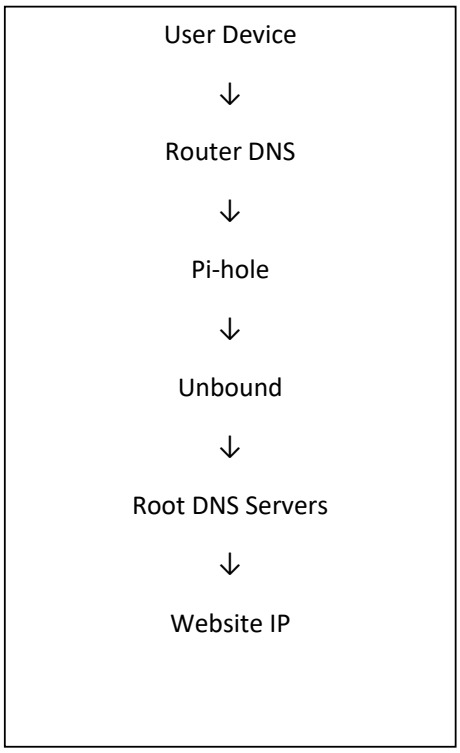
This eliminates port forwarding.

Benefits:

- safer
- encrypted
- zero trust

4.3 DNS Architecture

DNS architecture is central to security.



Step 1: Router DNS

Your router is configured to use Pi-hole as primary DNS.

Therefore every device automatically uses Pi-hole:

- phones
- TVs
- laptops
- tablets

No manual setup required.

Step 2: Pi-hole Filtering

Pi-hole checks whether domain belongs to:

- ads
- trackers
- telemetry
- malware

If blocked:
Request ends immediately.

Benefits:

- faster browsing
- less ads
- better privacy

Step 3: Unbound Resolution

If domain is safe, query goes to Unbound.

Unlike Google DNS or Cloudflare DNS, Unbound performs recursive resolution itself.

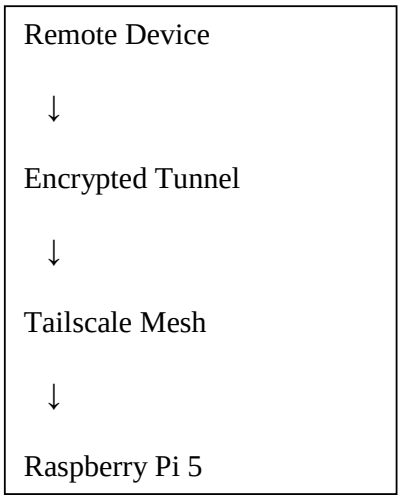
Advantages:

- privacy
- no third-party logging
- DNS independence

4.4 Remote Security Architecture

Remote access uses two independent security systems.

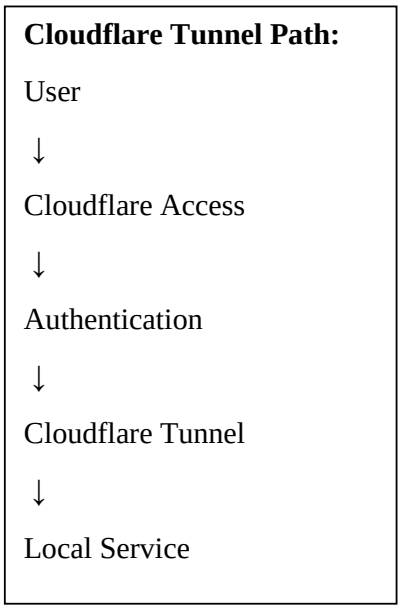
Tailscale VPN Path



Uses WireGuard encryption.

Ideal for:

- SSH
- terminal
- admin access



4.5 Cloudflare Access Control

Security policies:

Email Verification

Only authorized email accounts can access public URLs.

Unauthorized users are blocked before reaching NAS.

Session Timeout

Configured timeout:
30 minutes

After timeout:

- session invalidates
- re-auth required

This reduces hijacking risk.

Zero Trust Model

Trust is never assumed based on network location.

Every request is verified.

This dramatically improves security compared to traditional port forwarding.

4.6 Application Architecture

The application layer contains multiple services.

Media Services

- Jellyfin
- Navidrome

Storage Services

- NAS storage
- Syncthing

Backup Services

- Immich
- backup sync

Future Services

- Nextcloud
- MongoDB
- MariaDB
- Home automation

All applications run in isolated containers.

This prevents cascading failures.

Chapter 5 — CasaOS Implementation

5.1 Introduction to CasaOS

CasaOS is the primary operating dashboard used for managing the self-hosted NAS infrastructure. Although its name suggests a full operating system, CasaOS is actually a **server management platform** installed on top of Linux. It provides a web-based graphical interface that simplifies deployment and administration of self-hosted applications.

Traditional Linux server management often requires heavy command-line usage for:

- package installation
- service management
- storage mounting
- container deployment
- troubleshooting

For beginners or non-system administrators, this can become complex. CasaOS reduces that complexity by providing an intuitive UI where most daily tasks can be performed through the browser.

In this project, CasaOS acts as the **central control panel** for:

- application deployment
- Docker container monitoring
- storage access
- service management
- dashboard-based administration

It transforms the Raspberry Pi 5 from a command-line server into a user-friendly NAS appliance.

5.2 Why CasaOS Was Chosen

Several alternatives were considered before selecting CasaOS, such as:

- OpenMediaVault
- Portainer
- TrueNAS

CasaOS was selected because it offered the best balance between simplicity and functionality.

Reasons for Selection

Beginner-Friendly Interface

The UI is clean and easy to navigate.

Unlike enterprise dashboards, CasaOS avoids clutter and makes common operations straightforward.

Fast Deployment

Applications can be installed quickly using templates.

Lightweight

Raspberry Pi servers require efficient software. CasaOS consumes minimal resources.

Docker Integration

Since this project relies heavily on Docker, CasaOS provides a convenient visual layer over container management.

Storage Visualization

Mounted drives and folders are easily accessible.

5.3 CasaOS Installation

CasaOS installation on Raspberry Pi OS Lite is straightforward.

Step 1: Update System

sudo apt update && sudo apt upgrade -y

This ensures all packages are updated before installation.

Step 2: Install CasaOS

Official installation command:

curl -fsSL https://get.casaos.io | sudo bash

Installation process automatically configures:

- dependencies
- Docker (if missing)
- networking requirements
- CasaOS services

After installation, CasaOS becomes accessible through browser.

Example local access:

http://192.168.x.x

5.4 CasaOS Dashboard

After login, the dashboard serves as the central management screen.

Main dashboard sections include:

App Launcher

Shows installed applications as clickable icons.

Examples:

- Pi-hole
- Jellyfin
- Immich
- Navidrome

This resembles a desktop environment but runs in browser.

Resource Monitoring

Dashboard displays:

- CPU usage
- RAM usage
- disk utilization
- network activity

For your system:

- CPU idle ≈ 1%
- RAM idle ≈ 20%

This indicates efficient resource usage.

Storage Manager

The storage section helps manage:

- mounted SSD volumes
- directories
- NAS shares
- free capacity

This simplifies file management.

5.5 CasaOS App Store

One of CasaOS's strongest features is its App Store.

It enables one-click installation of popular self-hosted services.

Examples available:

- Jellyfin
- Immich
- Nextcloud
- Syncthing
- databases
- monitoring tools

Benefits of App Store:

Reduced Deployment Complexity

No need to manually configure every container from scratch.

Faster Experimentation

New services can be tested quickly.

Centralized Management

All apps appear in one dashboard.

5.6 File Browser

CasaOS includes built-in file browsing.

Capabilities:

- upload files
- download files
- move data
- create folders
- manage permissions

This allows basic NAS functionality without terminal usage.

Example directory structure:

```
/mnt/ssd/
├── media/
├── backups/
├── photos/
├── music/
└── docker/
```

This structured storage improves organization.

5.7 Service Management

CasaOS simplifies container lifecycle management.

Supported operations:

- start service
- stop service
- restart service
- update service
- remove service

Without UI, these tasks require multiple Docker commands.

CasaOS makes them accessible via buttons.

5.8 Benefits in This Project

In this NAS project, CasaOS provides:

Easy Administration

Services are visible in one place.

Lower Maintenance Overhead

Daily management becomes faster.

Better Monitoring

Resource tracking helps prevent overload.

Better User Experience

The server feels like a polished appliance rather than raw Linux.

Thus, CasaOS significantly improves practicality of the system.

Chapter 6 — Docker Architecture and Deployment

6.1 Introduction to Docker

Docker is the container runtime used to deploy nearly every service in this project.

A container packages:

- application code
- runtime
- dependencies
- libraries
- configuration

This ensures consistent execution across systems.

Instead of installing software directly on host OS, Docker isolates applications.

This is critical for a multi-service NAS.

6.2 Why Docker Was Used

Running services directly on Linux creates challenges:

Dependency Conflicts

Different applications may require different versions of libraries.

Example:
App A requires version X
App B requires version Y

This can cause breakage.

Docker isolates dependencies.

Easier Backup

Containers store persistent data in mapped volumes.

Example:

/docker/pihole
/docker/immich
/docker/jellyfin

Backing up these folders preserves configurations.

Fast Recovery

If container breaks:

1. Remove container
2. Redeploy image
3. Mount existing volume

Service returns quickly.

Portability

Containers can migrate to:

- new Raspberry Pi
- VPS
- x86 server

Minimal reconfiguration needed.

6.3 Docker Architecture in This Project

Docker stack includes multiple isolated containers.

Architecture:

Docker Engine
- ├── Pi-hole
- ├── cloudflared
- ├── Jellyfin
- ├── Immich
- ├── Navidrome
- ├── Syncthing
- └── Future Services

Each container performs dedicated tasks.

This modular design improves reliability.

6.4 Container Networking

Docker supports virtual networking.

Types include:

- bridge
- host
- macvlan

This project mainly uses bridge networking.

Benefits:

- isolation
- service communication
- port control

Example:

Pi-hole : 53
CasaOS : 80
Jellyfin: 8096
Immich : custom

Each service exposes required ports.

6.5 Docker Volumes

Containers are ephemeral by default.

Without persistent volumes:

- configuration resets
- media disappears
- databases vanish

Volumes solve this.

Example volume mappings:

```
volumes:
  - /mnt/ssd/media:/media
  - /mnt/ssd/config:/config
```

Benefits:

- persistence
- easier backup
- easier migration

6.6 Pi-hole Deployment

Example Docker deployment:

<YAML>

```
version: "3"

services:

  pihole:

    image: pihole/pihole:latest

    container_name: pihole

    restart: unless-stopped

    ports:

      - "53:53/tcp"

      - "53:53/udp"

      - "80:80/tcp"
```

Explanation:

Image

Downloads Pi-hole container.

Restart Policy

Automatically restarts after crash/reboot.

Port Mapping

Required for:

- DNS traffic
- web dashboard

6.7 Cloudflare Tunnel Deployment

Cloudflare tunnel container establishes secure outbound connection.

Example configuration:

<YAML>

services:

cloudflared:

image: cloudflare/cloudflared

command: tunnel run

Why containerized?

Benefits:

- isolated
- easy updates
- easier restart
- cleaner deployment

6.8 Tailscale Deployment / Installation

Tailscale can run either:

- directly on host
  or
- in container

Typical installation:

curl -fsSL https://tailscale.com/install.sh | sh

Authentication:

sudo tailscale up

After joining tailnet:
Pi gets secure private IP.

Example:

100.x.x.x

Now remote SSH becomes possible securely.

6.9 Jellyfin Deployment

Jellyfin provides media streaming.

Container requires access to:

- movies
- TV shows
- metadata
- subtitles

Example deployment:

<YAML>
services:
jellyfin:
image: jellyfin/jellyfin
volumes:
- /mnt/ssd/media:/media

Features enabled:

- library scanning
- metadata fetch
- remote streaming
- device playback

6.10 Immich Deployment

Immich is directly deployable in CasaOS after installing it from the CasaOS app store.

Requirements:

- storage
- database
- upload processing

Benefits:

- automatic mobile backup
- timeline gallery
- private photo cloud

6.11 Resource Utilization Analysis

One concern with Docker on SBCs is resource usage.

Observed idle state:

Metric	Value
CPU Usage	**~1%**
RAM Usage	**~20%**
Thermal State	**Stable**
Container Status	**Healthy**

This demonstrates Raspberry Pi 5 has substantial unused capacity.

Future services can still be added.

6.12 Advantages of Docker-Based Deployment

Main benefits achieved:

Isolation

Containers do not interfere.

Security

Reduced cross-service exposure.

Scalability

New services added easily.

Maintainability

Simpler updates and rollbacks.

Reliability

Crash in one container rarely affects others.

This architecture makes the NAS robust and future-ready.

Chapter 7 — Pi-hole and Unbound Implementation (DNS Security Architecture)

7.1 Introduction to DNS

Before understanding Pi-hole and Unbound, it is important to understand DNS.

DNS stands for **Domain Name System**. It acts like the internet's phonebook. Humans prefer domain names such as:

google.com
youtube.com
github.com

However, machines communicate using IP addresses:

142.250.x.x
172.217.x.x
140.82.x.x

DNS converts human-readable domain names into machine-readable IP addresses.

Whenever a user opens a website, DNS resolution happens first.

Example workflow:

User enters youtube.com
↓
DNS lookup starts
↓
IP address resolved
↓
Browser connects to server

Without DNS, users would need to remember numerical IP addresses for every website.

7.2 Problems with Traditional DNS Providers

Most users rely on DNS servers provided by:

- ISP (Internet Service Provider)
- Google Public DNS
- Cloudflare DNS
- OpenDNS

Although convenient, external DNS introduces concerns.

Privacy Concerns

Third-party DNS providers can see:

- visited domains
- request frequency
- browsing patterns
- device activity

Even if content is encrypted, DNS metadata reveals significant information.

Ads and Tracking

Many websites load:

- advertisements
- telemetry scripts
- analytics trackers
- malware domains

Standard DNS does not block them.

Result:

- slower browsing
- increased bandwidth
- reduced privacy

7.3 Introduction to Pi-hole

Pi-hole was selected because it offers:

Network-wide Protection

Unlike browser extensions, Pi-hole protects all devices.

Protected devices include:

- smartphones
- smart TVs
- laptops
- tablets
- IoT devices

Even devices without browser ad blockers benefit.

Centralized Filtering

One server manages DNS filtering for entire network.

No need to install software on every device.

Better Privacy

Tracking requests are blocked before data leakage.

Reduced Bandwidth Usage

Blocking ads reduces unnecessary network traffic.

Dashboard and Analytics

Pi-hole provides detailed statistics:

- total queries
- blocked queries

- top domains
- top clients
- blocked percentages

This improves visibility.

7.5 Router-Level Integration

In this project, router DNS is configured to use Pi-hole.

Architecture:

All Devices
↓
Router DNS
↓
Pi-hole

This design ensures automatic protection.

Benefits:

- zero per-device setup
- centralized administration
- consistent filtering

Every device joining Wi-Fi automatically uses Pi-hole.

7.6 Pi-hole Query Lifecycle

Let us examine a DNS request.

Suppose user opens:

example.com

Request flow:

Device
↓
Pi-hole
↓
Check block lists

Two possibilities exist.

Case 1: Domain Blocked

If domain belongs to ad/tracker list:

Blocked

Query stops immediately.

No connection is made.

Case 2: Domain Allowed

If safe:

Forward to upstream DNS

In this project, upstream DNS is Unbound.

7.7 Pi-hole Installation

Pi-hole can be deployed using Docker but in this case it is directly deployed in the pi os lite.

One step Automated Install:

curl -sSL https://install.pi-hole.net | bash

Container Example:

<YAML>

```
services:
  pihole:
    image: pihole/pihole
    container_name: pihole
    restart: unless-stopped
    environment:
      TZ: Asia/Kolkata
```

Port	Purpose
53 TCP	**DNS**
53 UDP	**DNS**
80 TCP	**Admin UI**

Admin dashboard accessible via:

http://pi-ip/admin

7.8 Pi-hole Dashboard Features

Dashboard provides useful analytics.

Total Queries

Shows DNS traffic volume.

High volume indicates heavy browsing activity.

Blocked Percentage

Shows effectiveness of filters.

Example:

Queries: 10000
Blocked: 3500
Block Rate: 35%

Top Clients

Identifies most active devices.

Examples:

- phone
- smart TV
- laptop

Top Blocked Domains

Shows ad/tracker domains.

Useful for security analysis.

7.9 Introduction to Unbound

Unbound is a validating recursive DNS resolver.

Traditional DNS flow:

User → Pi-hole → Google DNS → Internet

This still depends on third party DNS.

With Unbound:

User → Pi-hole → Unbound → Root DNS

No external DNS provider needed.

This improves privacy and control.

7.10 Recursive DNS Resolution

Recursive resolution means resolving domains directly from internet hierarchy.

DNS hierarchy:

1. Root Servers
2. TLD Servers
3. Authoritative Servers

Example resolution for:

github.com

Step-by-step:

Step 1: Ask Root Server

Root server says:
".com is managed here."

Step 2: Ask TLD Server

TLD server says:
"github.com authoritative server is here."

Step 3: Ask Authoritative Server

Returns final IP.

Unbound performs this automatically.

7.11 Benefits of Unbound

Privacy

No public DNS provider sees all queries.

Independence

No dependency on:

- Google DNS
- Cloudflare DNS
- ISP DNS

Cache Performance

Frequently requested domains get cached.

Benefits:

- lower latency
- faster browsing
- reduced repeated lookups

DNSSEC Validation

Unbound validates DNSSEC records.

This protects against:

- spoofing
- forged responses
- cache poisoning

7.12 Combined Pi-hole + Unbound Architecture

Combined flow:

Device

↓

Router

↓

Pi-hole

↓

Ad/Tracker Check

↓

Unbound

↓

Root DNS

↓

Internet

This architecture provides both:

Filtering

via Pi-hole

Privacy

via Unbound

Together they create a secure DNS pipeline.

7.13 Security Benefits Achieved

The DNS architecture improves security by:

- blocking malicious domains
- reducing trackers
- preventing phishing connections
- eliminating external DNS dependency
- enabling private recursive resolution

This significantly strengthens the NAS network.

Chapter 8 — Tailscale and Cloudflare Security Architecture

8.1 Need for Secure Remote Access

A NAS becomes far more useful when accessible remotely.

Remote access enables:

- file access
- media streaming
- terminal administration
- backup monitoring

However, exposing services to the internet creates risk.

Traditional remote access often uses:

- port forwarding
- public IP exposure
- open firewall ports

This increases attack surface.

Common risks:

- brute force attacks
- credential stuffing
- bot scanning
- exploit attempts

This project avoids those methods.

8.2 Tailscale VPN

Tailscale provides secure remote networking.

It uses **WireGuard-based encryption**.

Tailscale creates a private mesh network called a **tailnet**.

Devices join tailnet securely.

Example devices:

- phone
- laptop
- Raspberry Pi
- tablet

Each receives secure private IP.

Example:

100.x.x.x

8.3 Why Tailscale Was Chosen

Reasons include:

Easy Setup

Traditional VPNs require complex configuration.

Tailscale simplifies deployment.

Strong Encryption

Uses modern cryptography.

Traffic is encrypted end-to-end.

No Port Forwarding

Works without opening router ports.

Improves safety.

Mesh Connectivity

Devices communicate directly.

Low latency when peer-to-peer connection succeeds.

8.4 Remote SSH Access

One major benefit is terminal access.

Using Tailscale, remote SSH becomes simple.

Example:

ssh pi@100.x.x.x

This allows:

- container management
- troubleshooting
- system updates
- log inspection

Secure terminal access is critical for server maintenance.

8.5 Cloudflare Tunnel

Cloudflare Tunnel provides secure public access.

Traditional hosting:

Internet → Router Port Forwarding → NAS

This is risky.

Cloudflare Tunnel changes model:

NAS
↓
Outbound encrypted connection
↓
Cloudflare edge
↓
Public domain

No inbound port exposure.

This is a major security improvement.

8.6 Custom Domain Access

Benefits:

Better Accessibility

Instead of IP:

192.168.x.x:8096

Use:

example.domain.com

More professional and easier to remember.

HTTPS Support

Cloudflare provides TLS security.

Traffic remains encrypted.

8.7 Cloudflare Tunnel Working

Tunnel daemon (cloudflared) runs locally.

Example flow:

User
↓
Cloudflare DNS
↓
Cloudflare Edge
↓
Encrypted Tunnel
↓
Local Service

The service remains hidden from public internet.

Attackers cannot directly scan local ports.

8.8 Cloudflare Access (Zero Trust)

Cloudflare Access adds authentication.

This enforces Zero Trust.

Zero Trust principle:

Never trust by default. Always verify.

Every request must pass authentication.

8.9 Email Verification Policy

Access policy in your system:

- authorized emails allowed
- unknown emails denied

Authentication flow:

User requests service
↓
Cloudflare Access challenge
↓
Email verification
↓
Access granted

Unauthorized users never reach server.

8.10 Session Timeout

Configured timeout:

30 minutes

After timeout:

- session expires
- access token invalid
- login required again

Benefits:

- reduces hijacked sessions
- limits unauthorized persistence

8.11 Security Comparison

Method	Security
Port Forwarding	**Low**
Traditional VPN	**Medium**
Tailscale	**High**
Cloudflare Tunnel + Access	**Very High**

8.12 Combined Security Model

Layer 1

Pi-hole DNS filtering

Layer 2

Unbound recursive resolver

Layer 3

Tailscale encrypted VPN

Layer 4

Cloudflare Tunnel

Layer 5

Cloudflare Access authentication

This creates **defense in depth**.

Even if one layer fails, others remain active.

Chapter 9 — Application Services Hosted on NAS

9.1 Overview of Hosted Services

A major strength of this project is that the Raspberry Pi 5 server is not limited to basic storage. It acts as a multi-purpose self-hosted platform capable of running multiple applications simultaneously.

The application layer transforms the NAS into a complete personal cloud ecosystem.

Currently hosted or planned services include:

Service	Purpose
Jellyfin	**Media streaming**
Immich	**Photo backup**
Navidrome	**Music streaming**
Syncthing	**Backup replication**
Nextcloud	**Private cloud workspace**
MongoDB	**Development database**
MariaDB	**SQL database**

NOTE: These services run in isolated Docker containers, ensuring modularity and stability.

9.2 Jellyfin Media Server

Jellyfin is an open-source media streaming platform.

It allows the NAS to function like a personal version of commercial streaming platforms such as:

- Netflix
- Prime Video
- Disney+

Instead of streaming internet content, Jellyfin streams the user's personal media library.

Supported content includes:

- movies
- TV series
- anime
- documentaries
- lectures
- local video archives

9.2.1 Why Jellyfin Was Chosen

Several media servers exist:

- Plex
- Emby
- Jellyfin

Jellyfin was preferred because:

Open Source

No proprietary lock-in.

No Subscription

No premium license required.

Privacy Friendly

No unnecessary cloud dependency.

Self-Controlled Metadata

Library remains fully user-managed.

9.2.2 Jellyfin Working

Media files are stored inside SSD directories.

Example:

/media
├── Movies
├── TV Shows
├── Anime
└── Documentaries

Jellyfin scans the media folders and fetches metadata.

Metadata includes:

- posters
- thumbnails
- release year
- ratings
- cast information
- descriptions

This creates a polished media interface.

9.2.3 Streaming Workflow

Workflow:

User Device
↓
Jellyfin App/Web
↓
NAS Media Library
↓
Video Streaming

Supported devices:

- smartphone
- smart TV
- tablet
- browser
- laptop

9.3 Immich Photo Backup System

Immich is one of the most important services in this project.

Immich acts as a self-hosted alternative to:

- Google Photos
- Apple Photos

Its main purpose is automatic photo and video backup.

9.3.1 Need for Photo Backup

Modern smartphones generate huge media collections.

Examples:

- camera photos
- screenshots
- WhatsApp media
- videos
- recordings

Problems with cloud photo services:

- storage limits
- subscription fees
- privacy concerns

Immich solves these problems.

9.3.2 Immich Features

Key features include:

Automatic Upload

Mobile app uploads photos automatically.

Timeline View

Media shown chronologically.

Album Management

Photos grouped efficiently.

Search and Indexing

Fast media retrieval.

Local Ownership

All media stored on NAS.

9.3.3 Backup Flow

Photo backup architecture:

Phone Camera
↓
Immich Mobile App
↓
Encrypted Upload
↓
Raspberry Pi NAS
↓
SSD Storage

Benefits:

- automatic backup
- local ownership
- no third-party dependency

9.4 Navidrome Music Streaming

Navidrome is a self-hosted music server.

It provides Spotify-like personal music streaming.

Music formats supported:

- MP3
- FLAC
- AAC
- WAV

9.4.1 Why Navidrome

Advantages:

Lightweight

Very efficient on Raspberry Pi.

Fast Scanning

Music libraries index quickly.

Clean Interface

Simple and modern UI.

Remote Streaming

Music accessible anywhere.

9.4.2 Music Architecture

Example library:

/music

├── Artist 1

├── Artist 2

├── Albums

Workflow:

Music Library

↓

Navidrome Indexing

↓

Web / Mobile Access

↓

Streaming

This allows personal streaming without external subscriptions.

9.5 Syncthing Backup Architecture

Syncthing provides peer-to-peer synchronization.

Purpose:

- backups
- redundancy
- replication

Unlike cloud sync, data stays private.

9.5.1 Why Syncthing

Advantages:

Decentralized

No central cloud server.

Secure

Encrypted transfers.

Automatic Sync

Changes replicate automatically.

Cross Platform

Supports:

- Linux
- Windows
- Android
- macOS

9.5.2 Backup Flow

Primary NAS

↓

Syncthing Sync

↓

Backup Device

This improves disaster recovery.

9.6 Nextcloud (Future Scope)

Nextcloud is planned as future expansion.

Nextcloud provides:

- cloud storage
- file sharing
- notes
- calendar
- contacts
- collaborative workspace

It acts as private alternative to:

- Google Workspace
- Microsoft 365

9.7 Development Environment Support

NAS can also function as development infrastructure.

Possible self-hosted development services:

Databases

- MongoDB
- MariaDB

Web Hosting

Can host:

- websites
- APIs
- dashboards

Authentication

Possible to host:

- OAuth server
- private auth system
- SSO infrastructure

This demonstrates scalability of project.

Chapter 10 — Coding and Configuration Files

10.1 CasaOS Installation Commands

System preparation:

```
sudo apt update
sudo apt upgrade -y
```

Install CasaOS:

```
curl -fsSL https://get.casaos.io | sudo bash
```

Verify service:

```
systemctl status casaos
```

10.2 Docker Installation

Install Docker:

```
curl -fsSL https://get.docker.com | sh
```

Check version:

```
docker --version
```

Check running containers:

```
docker ps
```

10.3 Pi-hole Docker Compose

Example deployment:

<YAML>

```
version: "3"

services:

  pihole:

    image: pihole/pihole:latest

    container_name: pihole

    restart: unless-stopped

    environment:

      TZ: Asia/Kolkata

      WEBPASSWORD: strongpassword

    ports:

      - "53:53/tcp"

      - "53:53/udp"

      - "80:80/tcp"

    volumes:

      - ./etc-pihole:/etc/pihole
```

10.4 Unbound Configuration

Example configuration:

<CONF>

server:

verbosity: 1

interface: 127.0.0.1

port: 5335

do-ip4: yes

do-udp: yes

do-tcp: yes

Important settings:

Port 5335

Used as internal resolver port.

TCP + UDP Enabled

Supports DNS requests.

10.5 Tailscale Installation

Install:

curl -fsSL https://tailscale.com/install.sh | sh

Authenticate:

sudo tailscale up

Check status:

tailscale status

Remote SSH:

ssh pi@100.x.x.x

10.6 Cloudflare Tunnel Configuration

Install cloudflared:

sudo apt install cloudflared

Login:

cloudflared tunnel login

Create tunnel:

cloudflared tunnel create nas

Config file

Example tunnel configuration:

<YAML>

tunnel: tunnel-id
credentials-file: /home/pi/.cloudflared/file.json

ingress:
- hostname: jellyfin.domain.com
service: http://localhost:8096
- service: http_status:404

This maps public domain to local service.

10.7 Cloudflare Access Policy

Policy logic:

```
IF email ∈ allowed_users
  ALLOW
ELSE
  DENY
```

Additional setting:

Session duration:

30 minutes

Security benefit:
Even valid sessions expire automatically.

10.8 Jellyfin Docker Config

Example:

```
services:
  jellyfin:
    image: jellyfin/jellyfin
    ports:
      - "8096:8096"
    volumes:
      - /mnt/ssd/media:/media
      - /mnt/ssd/config/jellyfin:/config
```

10.9 Immich Storage Mapping

Example:

```
services:
  immich:
    volumes:
      - /mnt/ssd/photos:/usr/src/app/upload
```

This ensures uploads persist on SSD.

10.10 Backup Commands

Backup example using rsync:

```
rsync -av /mnt/ssd /backup-drive
```

Syncthing automates similar functionality.

Chapter 11 — Testing and Performance Analysis

11.1 Introduction to Testing

Testing is an essential phase of system development because it verifies whether the deployed NAS infrastructure performs reliably under real-world conditions. Since this project combines storage, networking, security, remote access, and multiple containerized services, testing was conducted across several dimensions to ensure stability and efficiency.

The major testing objectives were:

- verify service availability
- measure system resource utilization
- evaluate remote access performance
- analyze DNS response behavior
- validate security controls
- assess overall reliability

The testing process focused on practical day-to-day workloads rather than synthetic enterprise benchmarks, since the system is currently designed for **personal usage**.

11.2 Testing Environment

The test environment consisted of:

Component	Configuration
Server	**Raspberry Pi 5 (8GB)**
Storage	**SSD via NVMe-to-SATA HAT**
OS	**Raspberry Pi OS Lite**
Network	**Gigabit Ethernet**
Management	**CasaOS**
Containers	**Docker**
DNS Stack	**Pi-hole + Unbound**
Remote Access	**Tailscale + Cloudflare**
Use Case	**Personal NAS**

Client devices used for testing:

- Android smartphone
- Windows laptop
- Smart TV
- Browser clients

These devices simulate common real-world access scenarios.

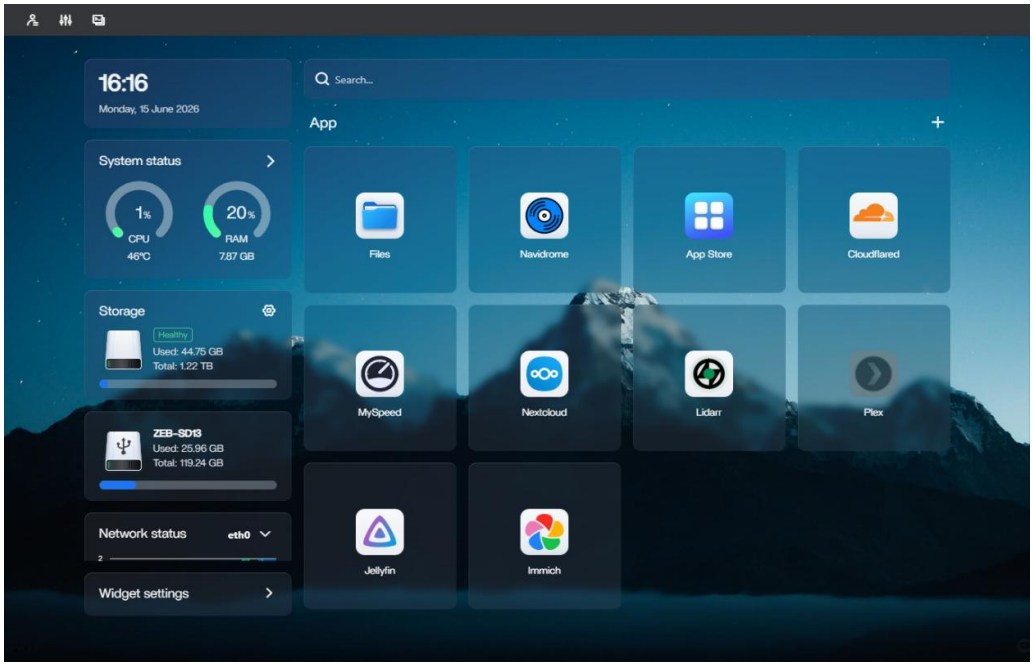
11.3 Functional Testing

Functional testing verifies whether each service behaves as expected.

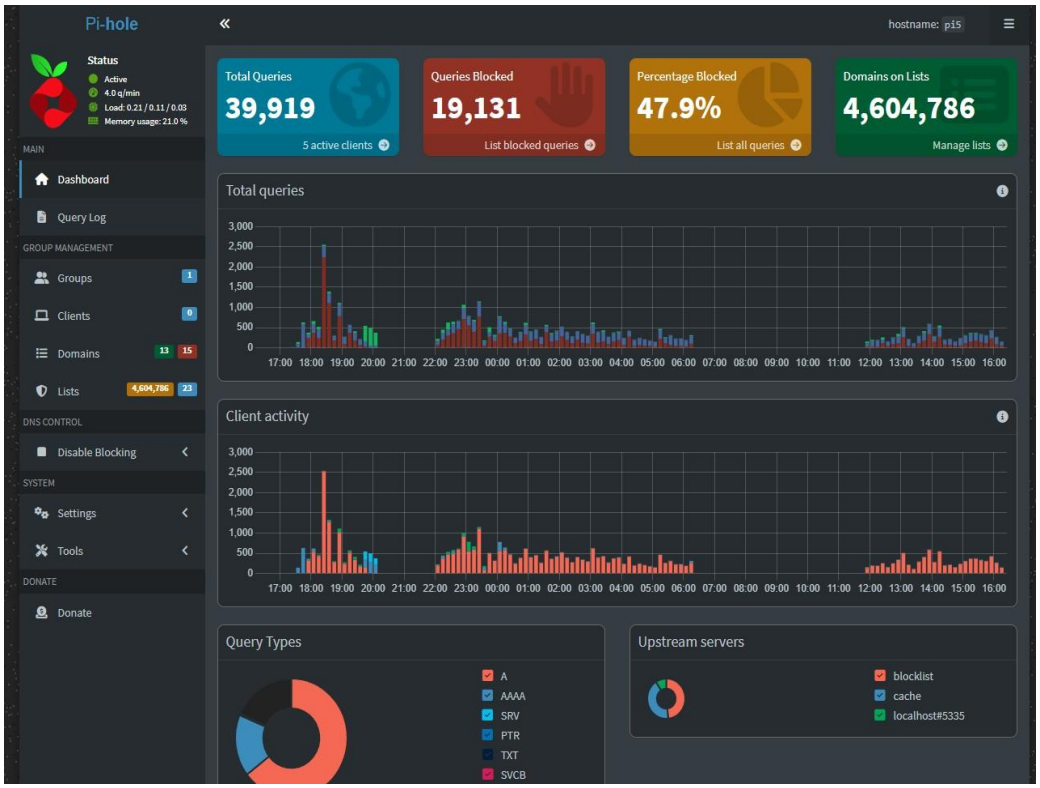
Test Case	Expected Result	Status
CasaOS UI access	**Dashboard opens**	**Pass**
Pi-hole DNS filtering	**Ads blocked**	**Pass**
Unbound recursion	**DNS resolves**	**Pass**
Tailscale remote SSH	**Successful login**	**Pass**
Cloudflare tunnel	**Remote access works**	**Pass**
Jellyfin streaming	**Smooth playback**	**Pass**
Immich backup	**Photos uploaded**	**Pass**
Speed Test	**Average speed test data**	**Pass**

11.3.1 Results of the testing

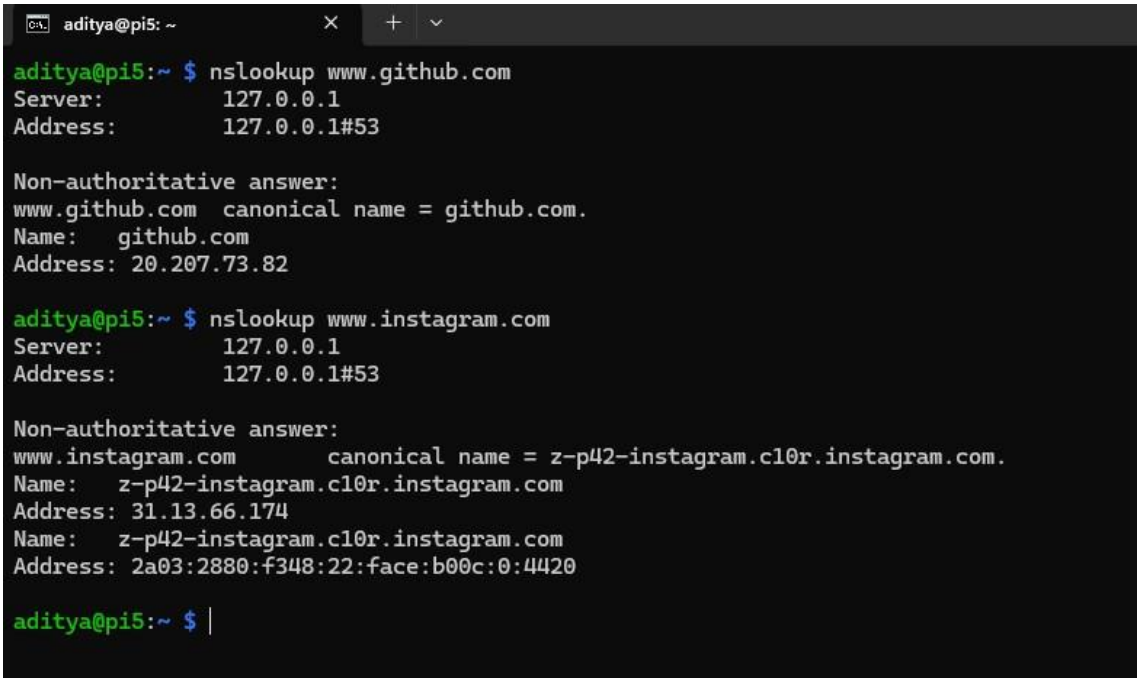
- Casa OS UI access



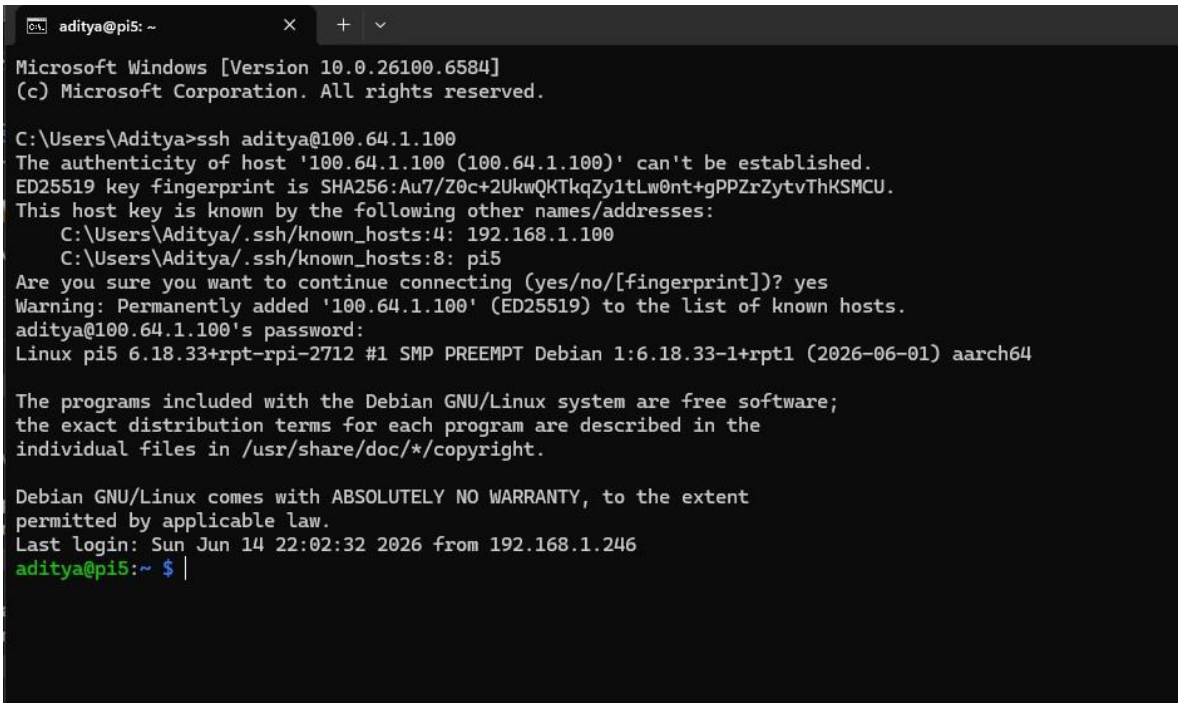
- Pi-hole DNS filtering



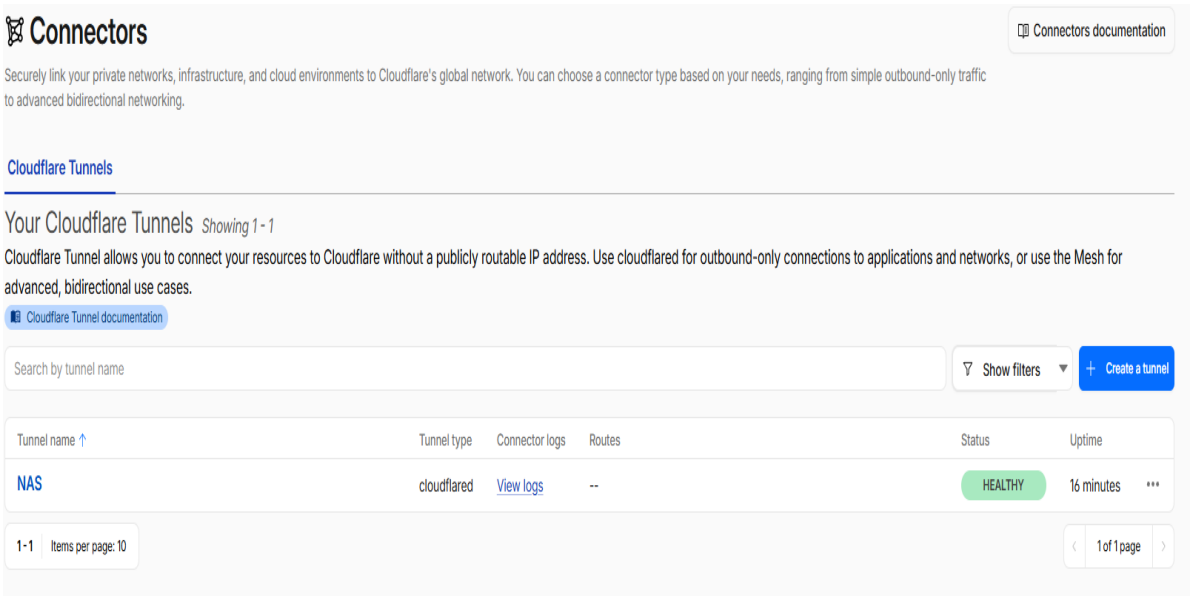
- Unbound recursion



- Tailscale remote SSH



- Cloudflare tunnel



- Network Monitoring

11.4 CPU Utilization Analysis

CPU usage indicates processing load on the server.

Observed idle CPU usage:

~1%

This is extremely efficient.

Reasons for low idle CPU usage:

- lightweight Linux OS
- Docker efficiency
- optimized services
- ARM power efficiency

Low CPU usage indicates significant available compute headroom.

This means future workloads such as:

- home automation
- databases
- monitoring tools
- additional services can still be added.

11.5 RAM Utilization Analysis

Memory utilization determines multitasking capability.

Observed RAM usage:

~20%

On an 8GB system, this means most memory remains available.

Approximate free memory:

6+ GB available

This demonstrates efficient resource allocation.

Services consuming RAM include:

- Docker engine
- CasaOS
- Pi-hole
- Jellyfin
- Immich
- background processes

Despite multiple services, memory pressure remains low.

11.6 Storage Performance Analysis

Storage performance directly impacts NAS responsiveness.

SSD storage significantly improves performance over microSD.

Key improvements:

Faster Boot Time

OS loads quickly.

Faster File Transfer

Large files transfer smoothly.

Examples:

- videos
- backups
- photos

Better Database I/O

Applications such as:

- Immich
- metadata services
- media indexing

benefit greatly from SSD.

Reduced Latency

Random reads/writes improve significantly.

This improves:

- UI responsiveness
- thumbnail generation
- indexing speed

11.7 Network Performance Testing

NAS systems depend heavily on network quality.

Testing parameters included:

- latency
- transfer reliability
- streaming performance
- DNS speed

Gigabit Ethernet ensures stable throughput.

Benefits observed:

Low Packet Loss

Stable connections during transfer.

Better Streaming

High bitrate videos stream smoothly.

Stable Remote Sessions

SSH and dashboard access remain reliable.

11.8 DNS Performance Testing

DNS speed affects browsing experience.

Testing performed on:

- domain resolution speed
- cache performance
- blocked requests

DNS request path:

Client
↓
Pi-hole
↓
Unbound
↓
Internet

Cold Resolution

First lookup requires recursive resolution.

Slightly slower than cached queries.

Cached Resolution

Repeated requests become significantly faster.

Advantages:

- lower latency
- faster browsing
- reduced internet dependency

11.9 Pi-hole Blocking Performance

Ad-blocking efficiency was evaluated.

Pi-hole successfully blocked:

- advertisements
- trackers
- telemetry domains
- malicious hosts

Benefits observed:

Cleaner Browsing

Reduced ad clutter.

Faster Page Load

Fewer unnecessary requests.

Privacy Improvement

Tracking domains blocked.

11.10 Jellyfin Streaming Test

Media playback tested across devices.

Content tested:

- 720p video
- 1080p video
- music playback

Performance observations:

Library Loading

Fast and responsive.

Metadata Retrieval

Posters and thumbnails load properly.

Playback

Smooth under normal conditions.

No major buffering issues observed during direct play.

11.11 Immich Backup Testing

Photo backup testing involved:

- image uploads
- video uploads
- repeated sync
- gallery refresh

Results:

Upload Success

Media uploaded correctly.

Indexing Success

Photos visible in timeline.

Automatic Backup

Works with mobile app.

This confirms reliability of private photo cloud.

11.12 Remote Access Testing

Two remote methods were tested.

Tailscale

Used for:

- SSH
- terminal access
- internal services

Results:

- secure connection established
- stable encrypted tunnel
- reliable remote admin

Cloudflare Tunnel

Used for:

- web service access
- dashboard access
- public domain routing

Results:

- domain routing successful
- services accessible remotely
- no direct port exposure

11.13 Reliability Testing

Long-duration testing examined system stability.

Key factors:

- uptime
- service persistence
- reboot recovery

After reboot:

- Docker restarted services
- CasaOS became accessible
- containers recovered automatically

This demonstrates good reliability for 24/7 operation.

11.14 Performance Summary

Metric	Result
Idle CPU	**~1%**
Idle RAM	**~20%**
Remote Access	**Stable**
DNS Filtering	**Effective**
Media Streaming	**Smooth**
Photo Backup	**Reliable**
Container Stability	**High**

Chapter 12 — Security Features and Threat Protection

12.1 Security Overview

Security is one of the most critical aspects of self-hosted infrastructure. Since the NAS stores personal files, media, configurations, and potentially sensitive information, multiple security layers were implemented.

The system follows **defense-in-depth architecture**, where security is distributed across several layers rather than relying on a single control.

Security layers include:

1. DNS security
2. VPN encryption
3. Zero Trust access
4. Tunnel security
5. Container isolation
6. Authentication policies

12.2 Threat Model

Potential threats considered:

- unauthorized access
- brute-force login attempts
- malware domains
- DNS poisoning
- network reconnaissance
- exposed service exploitation
- session hijacking

Each threat is mitigated through layered defenses.

12.3 DNS-Based Threat Protection

DNS is often the first step in malicious communication.

Attack chain example:

Device
↓
Malicious Domain
↓
Payload Delivery

Blocking malicious domains prevents later attack stages.

Pi-hole blocks:

- ad domains
- trackers
- phishing domains
- malware hosts

This reduces attack surface significantly.

12.4 Unbound Security Features

Unbound improves DNS integrity.

Security features include:

Recursive Resolution

No third-party resolver dependency.

DNSSEC Validation

Validates cryptographic DNS records.

Protects against:

- spoofed responses
- cache poisoning
- forged DNS answers

Cache Security

Improves consistency and efficiency.

12.5 Tailscale Encryption

Tailscale secures private access.

Built on WireGuard.

Benefits:

End-to-End Encryption

Traffic encrypted between peers.

Secure SSH

Terminal access remains protected.

Identity-Based Access

Only authenticated devices join network.

This prevents unauthorized devices from joining.

12.6 Risks of Port Forwarding

Traditional remote access exposes ports.

Example:

80
443
22

Attackers constantly scan internet for exposed ports.

Risks:

- brute force
- exploit attempts
- vulnerability scanning

This project avoids port forwarding entirely.

12.7 Cloudflare Tunnel Security

Cloudflare Tunnel eliminates inbound exposure.

Instead of opening router ports:

Server creates outbound encrypted connection.

Advantages:

Hidden Origin

Public users cannot directly see local IP.

Reduced Attack Surface

No exposed ports.

Encrypted Transit

Traffic remains protected.

This is safer than traditional reverse proxy setups.

12.8 Cloudflare Access Security

Cloudflare Access adds authentication before service access.

Access control policy:

- verify identity
- validate session
- enforce rules

Unauthorized requests blocked early.

12.9 Email-Based Authentication

Only authorized email accounts are allowed.

Flow:

Request
↓
Identity Check
↓
Allowed?
↓
Yes / No

Unknown identities are denied.

This prevents anonymous access.

12.10 Session Timeout Protection

Session duration:

30 minutes

Benefits:

- limits stolen session lifespan
- reduces persistent unauthorized access
- forces re-verification

This strengthens Zero Trust posture.

12.11 Docker Security Benefits

Containers improve security through isolation.

Example:

If one container crashes or is compromised:

- other containers remain separated
- host exposure reduces
- blast radius decreases

Benefits:

- process isolation
- filesystem isolation
- service separation

12.13 High Availability Planning

Raspberry Pi Zero 2W serves as planned backup node.

Potential failover services:

- Pi-hole
- Tailscale
- monitoring

Benefits:

- service continuity
- redundancy
- reduced downtime

12.14 Security Summary

Security Layer	Protection
Pi-hole	**Domain filtering**
Unbound	**Private DNS**
Tailscale	**VPN encryption**
Cloudflare Tunnel	**Hidden origin**
Cloudflare Access	**Zero Trust**
Docker	**Isolation**
Backups	**Recovery**

Chapter 13 — Future Scope

13.1 Introduction

Although the current NAS implementation already provides secure storage, remote access, media streaming, DNS security, and backup services, the architecture has been intentionally designed to remain scalable. One of the strongest advantages of using a modular Docker-based infrastructure on Raspberry Pi 5 is future expandability without requiring major architectural changes.

The current server uses only a portion of its available compute resources:

- CPU idle ≈ 1%
- RAM usage ≈ 20%

This indicates significant remaining capacity for future workloads. As user requirements grow, the system can evolve from a personal NAS into a complete self-hosted smart infrastructure platform.

Future improvements can be categorized into:

- automation
- scalability
- security enhancement
- development hosting
- AI integration
- high availability

13.2 Home Automation Integration

One of the most promising future expansions is **smart home automation**.

The NAS can be extended into a home automation controller using Home Assistant.

Home Assistant is a popular open-source automation platform that allows centralized control of smart devices.

Possible integrations include:

- smart lights
- smart plugs
- temperature sensors
- CCTV systems
- motion sensors
- door locks
- IR blasters
- energy meters

Automation Workflow

Example automation:

Motion Detected
↓
Night Time?
↓
Turn On Lights

This enables intelligent behavior inside the home.

Examples:

- auto light control
- appliance scheduling
- power monitoring
- security alerts

The Raspberry Pi NAS can become a unified smart-home hub.

13.3 Zigbee Device Support

Wireless smart devices often rely on communication protocols.

A highly efficient future upgrade is integration of Zigbee devices.

Advantages of Zigbee:

Low Power Consumption

Battery-operated sensors last longer.

Mesh Networking

Devices relay signals across network.

High Reliability

Better for automation than Wi-Fi in many scenarios.

Possible Zigbee devices:

- door sensors
- occupancy sensors
- switches
- smart bulbs

With a Zigbee USB coordinator, the NAS can control a complete automation ecosystem.

13.4 Advanced Monitoring and Analytics

The NAS can be enhanced with infrastructure monitoring tools.

Monitoring can include:

- CPU load
- RAM usage
- SSD health
- bandwidth usage
- ping latency
- uptime tracking
- internet stability

Potential tools:

- Prometheus
- Grafana

These tools can generate dashboards showing server health in real time.

Network Monitoring

Monitoring internet quality is useful for detecting instability.

Possible measurements:

- download speed
- upload speed
- packet loss
- ping latency
- jitter

Example periodic check:

Every 30 minutes
↓
Run speed test
↓
Log results

This helps identify ISP issues early.

13.5 Website Hosting

The current architecture already supports hosting websites.

Possible hosted services:

- personal portfolio
- blog
- landing pages
- business websites
- APIs

Since the server already uses:

- Docker
- Cloudflare Tunnel
- custom domain

web hosting can be added easily.

Possible stack:

- Nginx
- Node.js
- PHP
- static sites

This transforms NAS into a personal hosting server.

13.6 AI and Machine Learning Workloads

Self-hosted AI is rapidly becoming popular.

Possible future AI workloads:

- local LLM inference
- speech-to-text
- image classification
- personal assistant

Potential applications:

- smart chatbot
- voice assistant
- automation intelligence

Example workflow:

Voice Command

↓

Local AI Model

↓

Automation Trigger

This creates privacy-focused AI services without sending data to external clouds.

13.7 High Availability Expansion

Current high-availability planning includes Raspberry Pi Zero 2W.

Future improvements may include:

Active Failover

Automatic service switching.

Redundant Storage

Mirrored backups.

Load Distribution

Multiple nodes share workload.

Architecture example:

Primary Pi 5
↓
Failover Node
↓
Backup Storage

This reduces downtime during failures.

13.8 Cluster-Based Infrastructure

Future scaling may involve multiple nodes.

Example cluster:

- Pi 5 main node
- Pi Zero backup node
- secondary storage node

This can evolve into a mini homelab cluster.

Potential orchestration:

- Kubernetes
- k3s

Benefits:

- orchestration
- self-healing services
- scaling
- rolling updates

Chapter 14 — Conclusion

14.1 Project Summary

This project successfully demonstrates the design and implementation of a secure, scalable, and low-power **self-hosted NAS infrastructure using Raspberry Pi 5**. The system combines storage management, network security, remote accessibility, media services, backup systems, and cloud-like functionality into a compact yet powerful platform.

The project proves that enterprise-inspired infrastructure capabilities can be achieved using affordable consumer hardware when combined with efficient open-source software.

The implemented solution is far more than a simple file server. It functions as a complete personal cloud ecosystem capable of handling multiple real-world workloads simultaneously.

14.2 Major Achievements

The key objectives defined at the beginning of the project were successfully achieved.

NAS Implementation

A reliable storage system was created using SSD connected through NVMe-to-SATA expansion hardware.

This provided:

- fast storage access
- improved durability
- better I/O performance

compared to traditional SD-card-only setups.

Containerized Service Architecture

Using Docker enabled modular deployment of all services.

Benefits achieved:

- service isolation
- easier updates
- better recovery
- simpler maintenance

This created a scalable architecture.

User-Friendly Management

CasaOS greatly simplified system administration.

Through CasaOS, service management became easier for:

- monitoring
- deployment
- maintenance
- application installation

This improved overall usability.

Network Security Enhancement

A major achievement of the project was strong security implementation.

Security stack included:

- Pi-hole
- Unbound
- Tailscale
- Cloudflare Tunnel
- Cloudflare Access

Together these provided:

- DNS security
- ad blocking
- privacy
- encrypted access
- Zero Trust authentication

This significantly reduced attack surface.

14.3 Performance Evaluation

Performance testing showed excellent efficiency.

Observed system usage:

Metric	Observation
CPU Idle	**~1%**
RAM Usage	**~20%**
Service Stability	**High**
Remote Access	**Reliable**
DNS Performance	**Efficient**

These results indicate the Raspberry Pi 5 is highly capable for personal self-hosted workloads.

Even with multiple services active, the system maintained low resource usage.

This leaves substantial room for expansion.

14.4 Practical Benefits

The completed system provides real-world benefits.

Privacy

User data remains fully self-controlled.

Cost Savings

Reduces dependence on recurring cloud subscriptions.

Security

Multiple security layers protect infrastructure.

Flexibility

Open-source tools allow full customization.

Expandability

Future services can be added easily.

14.5 Limitations

Despite strong performance, certain limitations remain.

Limited Compared to Enterprise Servers

Raspberry Pi cannot match:

- multi-core Xeon servers
- enterprise storage arrays
- large virtualization clusters

Hardware Constraints

Heavy workloads such as:

- large AI models
- 4K transcoding
- large database clusters

may exceed SBC capability.

Single Main Node Dependency

Although failover planning exists, current architecture still relies mainly on one primary node.

Future redundancy improvements are desirable.

14.6 Final Conclusion

In conclusion, this project demonstrates that a modern **Raspberry Pi 5-based NAS** can serve as a powerful self-hosted infrastructure platform for personal usage. By integrating storage, networking, security, remote connectivity, media services, and backup solutions into one ecosystem, the system successfully achieves the goal of creating a secure and private alternative to centralized cloud services.

The project also highlights an important trend in modern computing: users increasingly value ownership, privacy, customization, and independence from proprietary ecosystems. Self-hosting enables all of these.

With future enhancements such as home automation, Zigbee integration, AI workloads, and multi-node clustering, this system has the potential to evolve into a complete smart digital infrastructure.

Therefore, the project can be considered a successful implementation of a **secure, scalable, and future-ready personal cloud architecture built on Raspberry Pi 5**.

BIBLIOGRAPHY / REFERENCES

Refrences

[1] Raspberry Pi Foundation, “Raspberry Pi 5 Documentation,” Raspberry Pi Official Documentation. Available: https://www.raspberrypi.com/documentation/

[2] Docker Inc., “Docker Documentation,” Docker Official Documentation. Available: https://docs.docker.com/

[3] CasaOS Community, “CasaOS Documentation,” CasaOS Official Documentation. Available: https://wiki.casaos.io/

[4] Pi-hole LLC, “Pi-hole Documentation,” Pi-hole Official Documentation. Available: https://docs.pi-hole.net/

[5] NLnet Labs, “Unbound Recursive DNS Resolver Documentation,” Unbound Documentation. Available: https://nlnetlabs.nl/projects/unbound/

[6] Tailscale Inc., “Tailscale Documentation,” Tailscale Official Documentation. Available: https://tailscale.com/kb/

[7] Cloudflare Inc., “Cloudflare Tunnel Documentation,” Cloudflare Docs. Available: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/

[8] Cloudflare Inc., “Cloudflare Access and Zero Trust,” Cloudflare Documentation. Available: https://developers.cloudflare.com/cloudflare-one/

[9] Jellyfin Project, “Jellyfin Documentation,” Jellyfin Docs. Available: https://jellyfin.org/docs/

[10] Immich Team, “Immich Documentation,” Immich Official Documentation. Available: https://immich.app/docs/

[11] Navidrome Contributors, “Navidrome Documentation,” Navidrome Docs. Available: https://www.navidrome.org/docs/

[12] Syncthing Foundation, “Syncthing Documentation,” Syncthing Docs. Available: https://docs.syncthing.net/

[13] Nextcloud GmbH, “Nextcloud Administration Manual,” Nextcloud Documentation. Available: https://docs.nextcloud.com/

[14] MongoDB Inc., “MongoDB Documentation,” MongoDB Official Docs. Available: https://www.mongodb.com/docs/

[15] MariaDB Foundation, “MariaDB Server Documentation,” MariaDB Documentation. Available: https://mariadb.com/kb/

[16] Home Assistant Community, “Home Assistant Documentation,” Home Assistant Docs. Available: https://www.home-assistant.io/docs/

[17] Kubernetes Authors, “Kubernetes Documentation,” Kubernetes Docs. Available: https://kubernetes.io/docs/

[18] Rancher Labs, “K3s Lightweight Kubernetes,” K3s Documentation. Available: https://docs.k3s.io/

[19] W. Richard Stevens, TCP/IP Illustrated, Volume 1: The Protocols, Addison-Wesley, 2011.

[20] Andrew S. Tanenbaum, Computer Networks, 5th Edition, Pearson Education, 2013.

[21] Behrouz A. Forouzan, Data Communications and Networking, McGraw-Hill Education, 2012.

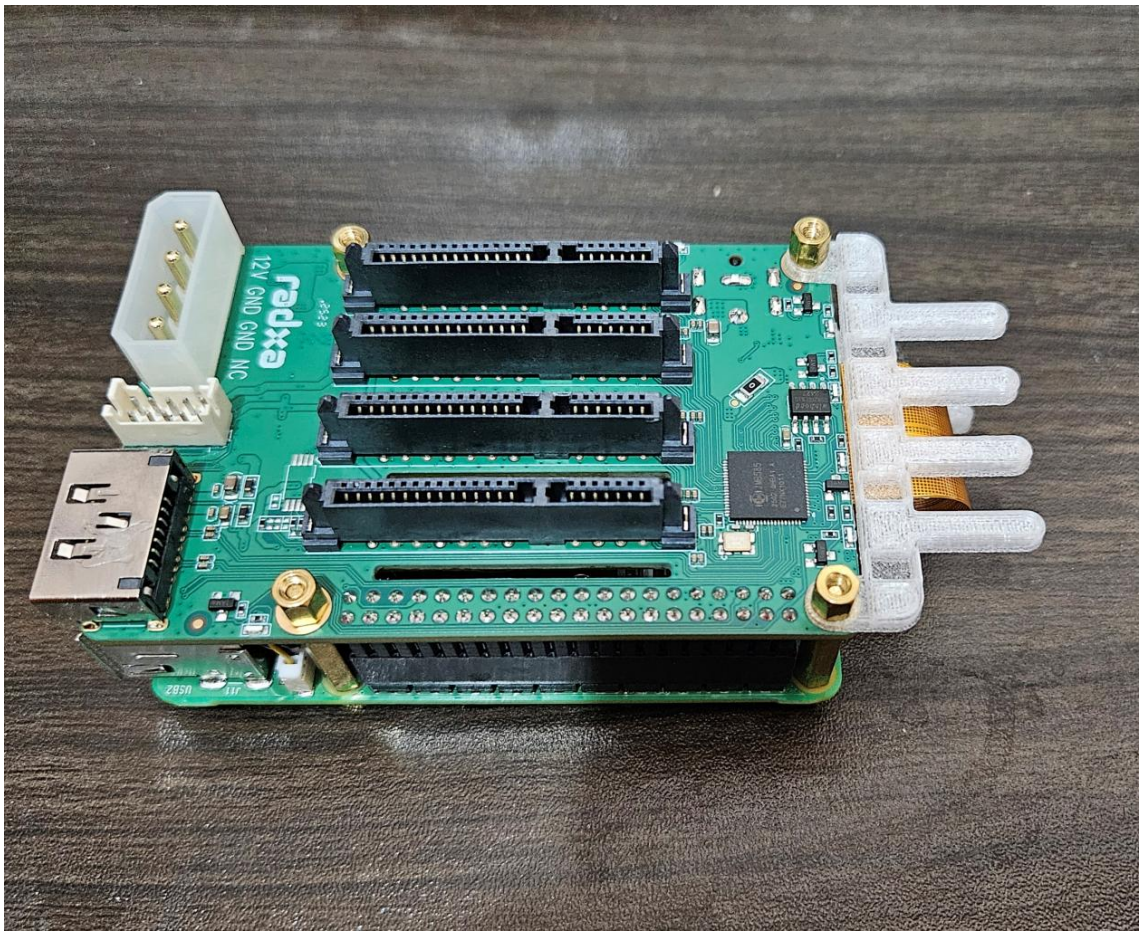
[22] William Stallings, Network Security Essentials: Applications and Standards, Pearson, 2017.

[23] Charles Severance, Linux System Administration, Open Textbook, 2020.

[24] J. Kurose and K. Ross, Computer Networking: A Top-Down Approach, Pearson, 2017.

[25] WireGuard Documentation, “WireGuard Protocol Overview.” Available: https://www.wireguard.com/

Hardware Images



WD Green™
SATA SSD
SOLID STATE DRIVE
480GB
Western Digital®